# Meeting 15. Unit II Essentials

A matrix expression carries several kinds of information at once: what its entries mean, how its shapes fit together, and how much work its evaluation requires. This review connects those views. The same habits help when reading a notebook, writing a numerical calculation, or explaining why an answer is reasonable.

For each problem, identify the objects, write the dimensions, inspect one output entry, and then consider cost and interpretation. A correctly shaped output can still answer the wrong question; a mathematically correct formula can still be too expensive to store.

**Companions.** The Unit II essentials slides provide the visual review. These notes consolidate Meetings 10-14. The worked examples below are practice examples; the Unit II Exam Prep module contains the course practice papers.

## The main points

1. **Interpret before multiplying**

   The two data Gram matrices compare different objects. A layer of neurons uses a different matrix whose rows are weight vectors. Sections 1-2.

2. **Track shapes through every step**

   Transpose, axis reduction, and broadcasting determine which quantities are combined. `*` and `@` perform different operations. Section 3.

3. **Count work and storage separately**

   A calculation may have manageable input size and an enormous output. Big O describes growth, while hardware and implementation determine measured seconds. Sections 4-5.

4. **A solution and a residual answer different questions**

   Rank concerns uniqueness. Conditioning concerns sensitivity. A residual checks the equations, and may not reveal a large solution error. Section 6.

5. **Sparsity helps only when the algorithm uses it**

   Store values and locations, account for indexing overhead, and distinguish a sparse matrix-vector product from a sparse solve. Section 7.

## 1. Data multiplied by data

Let $X$ have $n$ rows and $d$ columns. A row is one observation; a column is one feature. The dot product of two rows multiplies corresponding feature values and adds them. A matrix product performs many such dot products at once.

| Expression | Shape | Meaning of one entry |
|---|---|---|
| **Compare rows:** $XX^\top$ | $n \times n$ | Dot product of two observations |
| **Compare columns:** $X^\top X$ | $d \times d$ | Dot product of two feature columns |

The interpretation depends on the entries of $X$. If they are has-rated indicators, a dot product counts overlap. If they are star ratings, it sums products of ratings. If columns are centered and standardized with divisor $n$, dividing $X^\top X$ by $n$ gives column correlations. The multiplication alone does not choose the meaning.

Cosine similarity divides a dot product by the two vector lengths and requires both lengths to be nonzero. Correlation centers and standardizes the variables first. A missing observation should not silently become an observed zero. Specify the preprocessing whenever interpreting one of these comparisons.

### Two expressions that look similar

Suppose $X$ has shape $4 \times 3$. Then $XX^\top$ is $4 \times 4$, with three products added in each entry. The product $X^\top X$ is $3 \times 3$, with four products added in each entry. Both are symmetric, but their entries describe different pairs.

## 2. Data multiplied by neuron weights

A **neuron** is a rule that converts a feature vector into a score using weights and a bias, possibly followed by a nonlinear transformation. For a point $x$, its score is $w^\top x + b$. A layer applies several such rules to the same point.

Store one neuron's weights in each row of $W \in \mathbb{R}^{K \times d}$. There are $K$ neurons, each with one weight for every input feature. Applying them to all $n$ observations gives:

$$S = XW^\top + b, \qquad S \in \mathbb{R}^{n \times K}.$$

**In words:** The score matrix is the data matrix times the transpose of the weight matrix, plus the bias vector broadcast across rows. There is one score for each observation and each neuron.

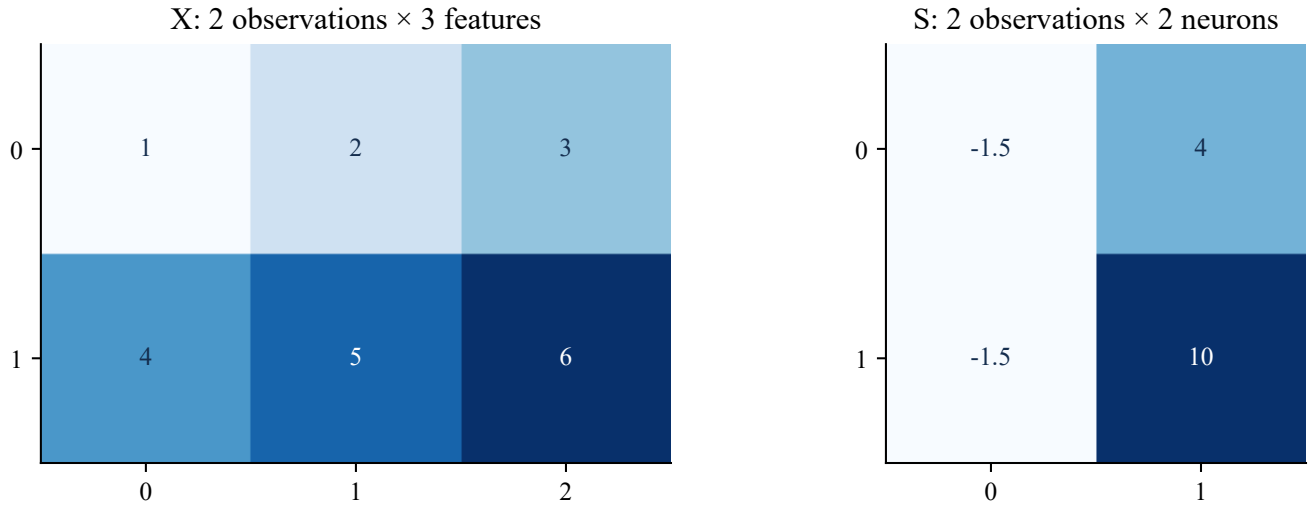
The bias vector has one entry per neuron. Adding it to every row gives each observation the same collection of neuron-specific biases. This operation is called **broadcasting**.

### A complete forward calculation

```
X = [[1, 2, 3],
     [4, 5, 6]]
W = [[1, 0, -1],
     [0, 1,  1]]
b = [0.5, -1]

X @ W.T + b = [[-1.5,  4],
               [-1.5, 10]]
```

The first neuron subtracts the third feature from the first, then adds 0.5. The second adds the last two features, then subtracts 1. Each row of the result belongs to one observation; each column belongs to one neuron.




Check the output by following one point through both rules. The repeated first-column score is a consequence of the selected weights and inputs, not an assumption about neurons.

## Scores, sigmoid, softmax, and a decision

A sigmoid converts one score to a number between 0 and 1: $\sigma(s) = 1 / (1 + e^{-s})$. Applying it independently to every entry does not generally make a row sum to one. Softmax instead normalizes a collection of scores for mutually exclusive alternatives:

$$P_{ij} = \frac{e^{S_{ij}}}{\sum_{\ell=1}^{K} e^{S_{i\ell}}}.$$

**In words:** Softmax divides the exponential of a score by the sum of exponentials of all scores in that observation's row.

Each softmax row sums to one. The largest probability has the same column index as the largest score, because exponentiation is increasing and the row shares a denominator. `argmax` returns that index; it does not return the probability or an original class label unless we translate the index with a label list.

### Stable row-wise softmax in NumPy

```
shifted = S - S.max(axis=1, keepdims=True)
E = np.exp(shifted)
P = E / E.sum(axis=1, keepdims=True)
prediction_index = P.argmax(axis=1)
assert np.allclose(P.sum(axis=1), 1)
```

Subtracting the same constant from every score in a row leaves its softmax probabilities unchanged. Using the row maximum makes every exponent nonpositive, avoiding overflow from large positive scores.

## 3. Read array expressions by their axes

An **axis** is a dimension of an array. A reduction removes or shortens the selected axis. In a two-dimensional data table, `axis=0` reduces over rows, leaving a result per column; `axis=1` reduces over columns, leaving a result per row.

| For `X.shape == (4, 3)` | Result shape | Meaning |
|---|---|---|
| `X[:, j]` | `(4,)` | Feature j across observations |
| `X[i, :]` | `(3,)` | The features of observation i |
| `X.mean(axis=0)` | `(3,)` | One mean per feature |
| `X.mean(axis=1)` | `(4,)` | One mean per observation |
| `X.mean(axis=1, keepdims=True)` | `(4, 1)` | The same row means, ready to broadcast across columns |

Broadcasting compares dimensions from the right. Each aligned pair must be equal, or one of the dimensions must be 1. A missing leading dimension behaves as 1. Thus `(4, 3) - (3,)` subtracts one value per column, while `(4, 3) - (4, 1)` subtracts one value per row. Subtracting `(4,)` fails here because the trailing dimensions 3 and 4 do not match.

> **A shape check is necessary, but not sufficient.**
>
> If a table happens to be square, a mistaken row-mean subtraction can broadcast successfully along columns. Check what one output entry should mean; accidental dimensional compatibility cannot verify intent.

For compatible arrays, `*` multiplies corresponding entries, with broadcasting where allowed. For two-dimensional arrays, `@` performs matrix multiplication: the left column count must equal the right row count. A one-dimensional NumPy array has no separate row and column axes, so its `.T` is unchanged. Use an explicit shape such as `(d, 1)` when a column dimension matters.

> **Standardizing columns**
>
> ```
> mu = X.mean(axis=0)
> sd = X.std(axis=0, ddof=0)
> # Handle constant columns before dividing.
> Z = (X[:, sd > 0] - mu[sd > 0]) / sd[sd > 0]
> ```
>
> The mean and standard deviation are both per-feature vectors. Every row uses the same feature means and scales. Standardizing each row would be a different transformation.

## 4. Count arithmetic and storage

A length-$d$ dot product uses $d$ multiplications and $d-1$ additions, approximately $2d$ scalar arithmetic operations. A matrix product has one such dot product per output entry. This is the conventional dense algorithm's count; an implementation may fuse a multiplication and addition into one instruction.

| Product | Output entries | Approximate arithmetic |
|---|---|---|
| Row Gram matrix: $XX^\mathsf{T}$ | $n^2$ | $2n^2d$ |
| Column Gram matrix: $X^\mathsf{T}X$ | $d^2$ | $2nd^2$ |
| Neuron scores: $XW^\mathsf{T}$ | $nK$ | $2nKd$ |

**Big O** suppresses constant factors and lower-order terms to describe growth. It is useful for comparing scaling, not for predicting the exact number of seconds. State what changes in a scaling question: doubling only $n$ differs from doubling all dimensions.

### A product that is larger than its input

A table of 50,000 word vectors with 300 coordinates has 15 million entries. Stored as float32, its values occupy 60 million bytes, or 60 MB in decimal units. Its full row Gram matrix has 2.5 billion entries: **10 GB** at float32, before array overhead or temporary storage.

The conventional product requires approximately $2 \times 50,000^2 \times 300 = 1.5 \times 10^{12}$ arithmetic operations. A calculation can be fast enough arithmetically yet fail because the output does not fit in memory.

If every matrix dimension doubles, the dense matrix-product work grows by a factor of eight and the output storage by a factor of four. If only the number of observations doubles, $XX^\mathsf{T}$ instead has four times the work and storage, while $X^\mathsf{T}X$ has twice the work and the same output shape.

## 5. Why equal operation counts take different times

Python loops pay interpreter overhead on each iteration. A vectorized NumPy operation moves a loop into compiled code; it may also use optimized libraries and multiple processor cores. Vectorization changes where and how the work is performed. It does not make a required arithmetic operation disappear.

Processors reuse nearby or recently accessed values through caches. Traversing contiguous memory and reusing blocks can reduce transfers from slower memory. Two programs with the same operation count may therefore move very different amounts of data, or move it with very different efficiency.

**Throughput** measures arithmetic per second; **bandwidth** measures bytes transferred per second. Arithmetic intensity is operations per byte moved. A workload with little reuse may be limited by memory traffic, while a dense product that reuses blocks can do much more arithmetic per transferred byte. Counting only arithmetic misses this distinction.

A GPU is a processor with its own execution costs and usually its own memory. Transferring inputs, launching work, and collecting outputs take time. Small operations may not repay those costs. A large sequence that reuses data already on the GPU can behave differently from repeatedly copying small arrays.

> **Report what a timing includes.**
>
> State the input size, numeric type, device, and whether transfers are included. GPU work can be asynchronous, so a timer around the launch may stop before the calculation finishes. Synchronize around the measured work when measuring completed GPU computation. Repeat measurements and separate setup from the work whose speed you are claiming.

## 6. Linear systems: equations, uniqueness, and accuracy

A linear system collects relationships between unknown quantities into $Ax = b$. Rows correspond to equations, columns to unknowns, and $b$ contains known contributions. Writing down the meaning of one row is often the best way to check the entire system.

### Random-walk probabilities give average equations

Consider a path with known endpoint values 1 and 0 and two interior nodes, $u$ and $v$. From an interior node, a walk chooses either neighbor with equal probability. Let each unknown be the probability of reaching the 1 endpoint before the 0 endpoint.




Conditioning on the first step makes each interior probability the average of its neighbors. The endpoint values are given; only u and v are unknown.

$$p_u = \frac{1 + p_v}{2}, \qquad p_v = \frac{p_u}{2}.$$

**In words:** The probability at u is one half of one plus the probability at v. The probability at v is one half of the probability at u.

Multiply by two, move unknown terms left, and retain known values on the right. The system is $2p_u - p_v = 1$ and $-p_u + 2p_v = 0$. Substitution or elimination gives $p_u = 2 / 3$ and $p_v = 1 / 3$. They lie between the endpoint values, as probabilities should.

**Solve and check the random-walk system**

```
A = np.array([[2., -1.], [-1., 2.]])
b = np.array([1., 0.])
p = np.linalg.solve(A, b)
residual = b - A @ p
print(p)                    # [0.66666667, 0.33333333]
print(np.linalg.norm(residual))
```

Use a solver for $Ax = b$ directly. Forming `np.linalg.inv(A) @ b` computes an entire inverse that the task does not require and is generally a less appropriate numerical route.

## Rank and conditioning are different

A square full-rank matrix has one solution for every right-hand side. A rank-deficient square matrix has either no solution or infinitely many, depending on consistency. A nearly rank-deficient matrix may still have a unique solution while being sensitive to small perturbations.

A **condition number** quantifies possible sensitivity in a specified norm. A large value indicates that some small relative changes in the input can produce large relative changes in the solution. It does not mean every perturbation produces the worst possible error, and it is a property of the mathematical problem rather than merely a description of the solver.

## A residual checks equations, not the unknown true answer

For a computed approximation $\hat{x}$, the residual is $r = b - A\hat{x}$. We can compute it from the supplied problem. The solution error is $x - \hat{x}$, which requires knowing the true solution. Because $A(x - \hat{x}) = r$, an ill-conditioned matrix can transform a substantial error into a tiny residual.

### A small residual with a large error

Take $A = \operatorname{diag}(1, 10^{-8})$ and $b = (1, 10^{-8})^{\mathsf{T}}$. The exact solution is $x = (1, 1)^{\mathsf{T}}$. For the guess $\hat{x} = (1, 0)^{\mathsf{T}}$, the residual is $(0, 10^{-8})^{\mathsf{T}}$, with length $10^{-8}$. The error is $(0, 1)^{\mathsf{T}}$, with length 1.

The second coordinate matters little to the equations' residual because its coefficient is tiny. The matrix has 2-norm condition number $10^{8}$.

Scale also matters: a residual should be assessed relative to the sizes of the problem data, not by a universal absolute threshold. For a nonsingular matrix and nonzero right-hand side, a standard normwise bound relates relative solution error to the condition number times relative residual. A large condition number can make that bound uninformative even when the residual is small.

## 7. Sparse storage and sparse algorithms

A sparse matrix has relatively few nonzero entries. Dense storage reserves space for every position. Sparse formats store nonzero values and enough indexing information to locate them. **COO** stores row, column, and value triples. **CSR** groups entries by row and uses a row-pointer array; **CSC** groups them by column.

For an $N \times N$ CSR matrix with $q$ stored nonzeros, a typical layout contains $q$ values, $q$ column indices, and $N + 1$ row pointers. Thus memory is proportional to $q + N$, with constants determined by the numeric and index types. Index overhead means sparse storage is not automatically smaller when many entries are nonzero.

### A storage estimate with stated assumptions

Let $N = 1000$ and $q = 5000$. With float64 values and 32-bit indices, CSR stores approximately `5000×8 + 5000×4 + 1001×4 = 64,004` bytes for those arrays. Dense float64 storage needs `1000×1000×8 = 8,000,000` bytes. These counts omit object overhead and temporary work arrays; using 64-bit indices changes the sparse count.

A CSR matrix-vector product visits the stored entries and produces one output per row, for $O(q + N)$ work. A *solve* is a different operation. Direct elimination can create new nonzero entries, called **fill-in**; eliminating a node in a graph can connect its remaining neighbors. The elimination order affects the amount of fill and therefore memory and work.

An iterative method repeatedly improves an approximation. Its total cost is the cost of one update times the number of updates, together with setup and stopping checks. Fast matrix-vector products help, but convergence can still be slow. Sparsity, conditioning, ordering, and the chosen algorithm all matter.

## 8. Practice the explanation as well as the calculation

For each question, write the dimensions or one defining equation before calculating. Then add one sentence interpreting the result.

#### 1. A layer has 8 neurons and receives 20 points with 5 features. What are the shapes?

$X$ is $20 \times 5$, $W$ is $8 \times 5$, and $W^{\mathsf{T}}$ is $5 \times 8$. The score matrix is $20 \times 8$. A bias vector with 8 entries broadcasts across the 20 observations.

#### 2. What changes if both dimensions of a data table double?

The input storage grows fourfold. Either Gram matrix has four times as many output entries. Each output dot product is twice as long, so conventional product work grows eightfold. This is an operation-count statement, not a timing guarantee.

### 3. Is a small residual proof of an accurate solution?

No. The residual checks how well the approximation satisfies the supplied equations. Conditioning determines how strongly that controls solution error. The diagonal example in Section 6 has residual length $10^{-8}$ and error length 1.

### 4. Does a sparse input guarantee a cheap direct solve?

No. Elimination can create fill-in. The nonzero pattern, elimination order, and factorization determine the eventual storage and arithmetic. The original nonzero count alone is insufficient.

### 5. Does a faster GPU kernel guarantee a faster complete calculation?

No. Compare the whole requested operation, including any transfers and setup it requires. Reusing data already on the GPU can change the answer. Timing only an asynchronous launch does not measure completed work.

## Return to the detailed notes

For the full examples, see Data as Arrays, Neurons as Matrices, Big O and the Machine, Solving Linear Systems, and Exploring Data with Matrix Products. The practice here draws on those topics and introduces no new exam rules.